LangChain

RAG

Cheatsheet

Sachin Patil

Data Loading

Purpose Load documents from various file formats so they can be processed by your pipeline.

Common Loaders:

- PyPDFLoader for PDFs
- TextLoader for plain text files

  CSVLoader for CSV data

```python
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("file.pdf")
documents = loader.load()
```

Sachin Patil

Document Chunking

Purpose **Split documents into smaller, manageable chunks to improve retrieval accuracy.**

Common Splitter:

- **RecursiveCharacterTextSplitter**

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
chunks = splitter.split_documents(documents)
```

Embeddings

Purpose **Convert text chunks into numerical vector representations that can be stored and searched**

Popular Embeddings:

- **OpenAIEmbeddings**

  HuggingFaceEmbeddings

```
from langchain.embeddings import OpenAIEmbeddings
embeddings = OpenAIEmbeddings()
```

Vector Stores

Purpose Store and retrieve vector embeddings

Popular Options:

- **Chroma, FAISS, Pinecone, Weaviate**

```
from langchain.vectorstores import FAISS
vectorstore = FAISS.from_documents(
    chunks, embeddings
)
```

Retrieval

Purpose **Convert a vector store into a retriever and fetch relevant document chunks based on user queries.**

```
retriever = vectorstore.as_retriever(
    search_type="similarity", k=3
)
docs = retriever.invoke("What is LangChain?")
```

Sachin Patil

RAG Pipeline

Purpose **Combine retrieval with a language model to answer questions based on retrieved context (Retrieval-Augmented Generation).**

Component:

- **RetrievalQA**

```
from langchain.chains import RetrievalQA
qa = RetrievalQA.from_chain_type(
    llm,
    retriever=retriever,
    chain_type="stuff"
)
result = qa.run("Explain RAG")
```

Sachin Patil